按插件左侧的树从上往下看，你这份 JMeter 计划的顺序是：**设置线程数和轮数 → 每个线程登录 → 读取本轮操作 → 等线程到齐 → 发请求 → 检查响应和数据库结果。** 下面每一段都可以对着 GUI 用俄语讲。

## 1. 线程组：C01、C02、C03 的参数从哪来？

**在 GUI 看** 展开线程组 `${__P(scenario,C02)}`。右侧的线程数是 `${__P(threads,2)}`。轮数在它下面的 **Fresh target per round**。

**中文** 三个场景共用一份 JMX。Python 启动时传入场景编号、线程数和轮数。C01 和 C03 各有 2 个线程，C02 有 10 个，完整测试每组跑 20 轮。`${__P(threads,2)}` 里的 `2` 是没有传线程数时的默认值。外层循环执行一次，所以每个线程只登录一次。内层循环才决定业务请求执行多少轮。

**Русский** Один файл JMX используется для трёх сценариев. При запуске Python передаёт номер сценария, число потоков и число раундов. В C01 и C03 по два потока, в C02 — десять. В полном прогоне каждый сценарий выполняется двадцать раундов. Число `2` в выражении `${__P(threads,2)}` — значение по умолчанию. Внешний цикл выполняется один раз, поэтому каждый поток входит в систему один раз. Внутренний цикл задаёт число раундов.

线程数定义 · C02 的实际启动参数

## 2. 登录：`Load private credentials` 做什么？

**在 GUI 看** 在线程组下找 **Load private credentials**，它在同一级的 **Login** 上面。点开后，右侧脚本文件指向 `jmeter/init.groovy`。再展开 **Login**，下面有 **Extract CSRF**。

**中文** `Load private credentials` 给当前线程选一个测试账号，并准备登录请求体。下面的 `Login` 才真正发送 `POST /auth`。Cookie Manager 给每个线程保留自己的会话，`Extract CSRF` 从登录响应里取出 token。后续写请求带着这个会话 Cookie 和 `X-CSRF-Token`。

**Русский** `Load private credentials` выбирает тестовую учётную запись для текущего потока и подготавливает тело запроса. Затем `Login` отправляет `POST /auth`. Cookie Manager хранит сеанс каждого потока отдельно, а `Extract CSRF` извлекает токен из ответа. Последующие запросы записи отправляют cookie сеанса и заголовок `X-CSRF-Token`.

JMX 登录部分

## 3. 每轮取数据：`Prepare shared fixture` 和 `fixture.json` 是什么？

**在 GUI 看** 展开 **Fresh target per round** → **Prepare shared fixture**。这是循环里的第一个节点，右侧指向 `jmeter/prepare.groovy`。

**中文** `fixture.json` 是 Python 为本次测试生成的**操作清单**。脚本根据「当前轮次 + 当前线程编号」选一条 `action`，再把方法、路径、请求体或文件路径填进 JMeter 变量。以 C03 为例，操作清单写着两个线程都发 `POST imports`，但一个上传 `r0-p0.json`，另一个上传 `r0-p1.json`。这两份 `r0-p*.json` 里面才是发给应用的电影数据。

**Русский** `fixture.json` — это список действий, который Python готовит для текущего прогона. Скрипт выбирает действие по номеру раунда и номеру потока. Из него он берёт HTTP-метод, путь, тело запроса или путь к файлу. Например, в C03 оба потока отправляют `POST /imports`. Первый загружает `r0-p0.json`, второй — `r0-p1.json`. Данные фильмов находятся в этих двух файлах.

线程怎样选操作 · C03 第一轮操作清单

## 4. 并发请求：具体操作在哪个节点?

**在 GUI 看** 先指 **Read initial version** 和 **Bounded rendezvous**，再展开两个目标请求分支。

| 点哪个节点 | 中文 | Русский |
|---|---|---|
| **Read initial version → Read same target version** | C01 的两个线程先 `GET` 同一部电影，确认拿到相同的初始版本。 | В C01 оба потока сначала читают один фильм через `GET` и проверяют, что получили одну и ту же исходную версию. |
| **Bounded rendezvous → Synchronizing Timer** | 同一轮的线程在这里等彼此到齐，再尽量同时发出写请求。 | Потоки одного раунда ждут друг друга на барьере, чтобы отправить запросы записи почти одновременно. |
| **Create, edit or delete → `${__P(scenario)} target`** | 右侧显示 `${targetMethod}`。C01 给一个线程填 `PUT` 编辑，另一个填 `DELETE` 删除。C02 给所有线程填 `POST`，并用相同的电影名称和导演 ID。 | Метод подставляется в `${targetMethod}`. В C01 один поток отправляет `PUT`, другой — `DELETE`. В C02 все потоки отправляют `POST` с одинаковым названием |

| 点哪个节点 | 中文 | Русский |
| --- | --- | --- |
| | | фильма и одним ID режиссёра. |
| **Multipart import →** `${__P(scenario)} import` | C03 固定发 `POST /imports`。在右侧**文件上传**里，`${uploadFile}` 是各线程自己的文件路径，`file` 是上传字段名。 | В C03 запрос отправляется методом `POST` на `/imports`. Переменная `${uploadFile}` указывает на файл текущего потока, а `file` — имя поля загрузки. |

普通写请求 · 上传请求

## 5. 检查结果：为什么 `409` 也能显示通过？

**在 GUI 看** 展开目标请求下面的 **Business status and raw result**，右侧指向 `jmeter/record.groovy`。

**中文** `record.groovy` 检查当前响应是否符合这个操作的预期。比如 C01 的归档第一轮，编辑返回 `200`，删除因为旧版本返回 `409`。两条都符合预期，所以都显示 `passed: true`。整个场景结束后，Python 的 `verify()` 再检查最终数据。C01 要确认新标语和版本确实已保存，或者电影确实已被删除。C02 要确认每轮只保存一部匹配的电影。C03 要确认失败文件的第一条也被回滚，数据库里只留下成功文件的数据。

**Русский** `record.groovy` проверяет, соответствует ли конкретный ответ ожидаемому результату. Например, в первом сохранённом раунде C01 редактирование получило `200`, а удаление со старой версией — `409`. Оба ответа ожидаемые, поэтому у обоих стоит `passed: true`. После завершения сценария функция `verify()` проверяет итоговые данные. В C01 она проверяет новый слоган и версию либо отсутствие фильма. В C02 она проверяет, что в каждом раунде сохранился только один фильм с нужным ключом. В C03 она проверяет, что откат затронул и первую запись отклонённого файла, а в базе остались только данные успешного импорта.

C01 第一轮实际响应 · Python 最终核对